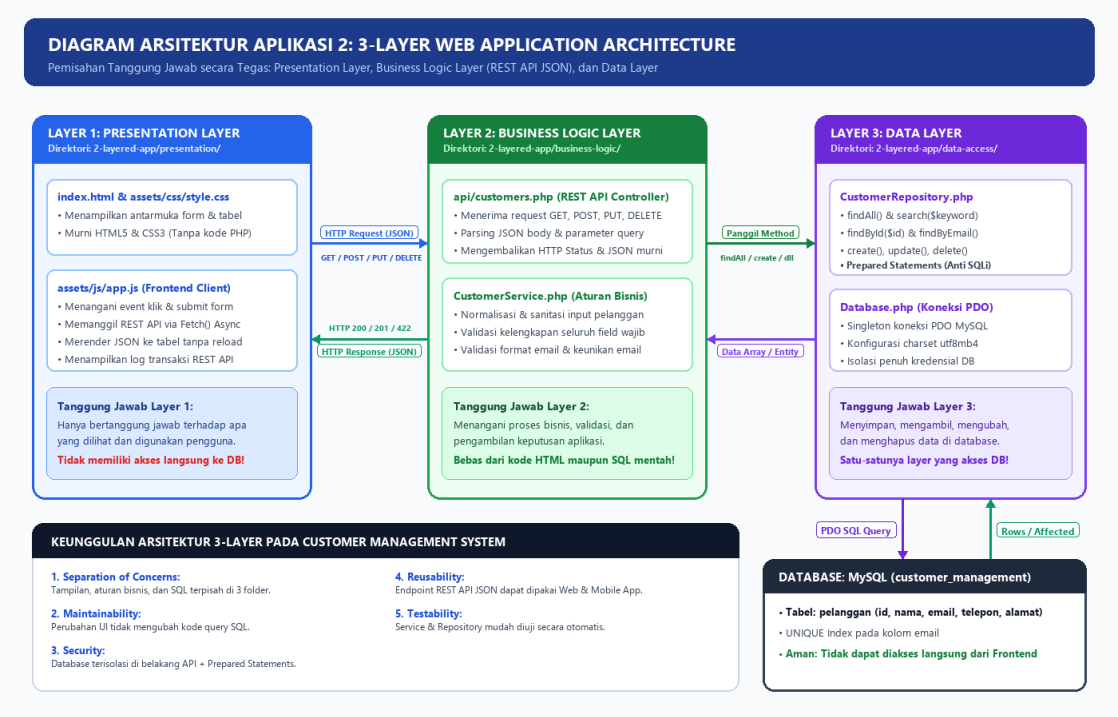
LAPORAN TUGAS KELOMPOK

ANALISIS DAN IMPLEMENTASI PERBANDINGAN LAYERED VS NON-LAYERED WEB APPLICATION ARCHITECTURE

Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)



Mata Kuliah	Pemrograman Web / Arsitektur Aplikasi Web
Topik Kajian	3-Layer Web Application Architecture vs Non-Layered Monolithic Architecture
Studi Kasus	Customer Management System (CRUD, Pencarian Nama/Email, Validasi Kelengkapan & Keunikan Email)
Teknologi	PHP Native (Non-Framework), MySQL (PDO), HTML5, CSS3, JavaScript Fetch API (REST & JSON)

BAB 1. PENDAHULUAN DAN LANDASAN TEORI

Dalam pengembangan aplikasi web modern, pemilihan arsitektur perangkat lunak sangat menentukan kemudahan pemeliharaan (*maintainability*), keamanan (*security*), skalabilitas (*scalability*), serta kemampuan kode untuk digunakan kembali (*reusability*). Laporan ini menganalisis dan membandingkan secara langsung dua pendekatan arsitektur pengembangan aplikasi web melalui pembangunan sistem yang identik secara fungsional, yaitu **Customer Management System**.

1.1 Konsep Non-Layered Web Application Architecture (Monolitik Sederhana)

Non-Layered Architecture (sering disebut aplikasi monolitik satu file) adalah pendekatan pengembangan di mana kode antarmuka pengguna (HTML/CSS), logika proses bisnis (validasi dan aturan aplikasi), serta akses basis data (koneksi dan query SQL) ditempatkan bersama di dalam satu file atau beberapa file tanpa pemisahan tanggung jawab yang tegas. Pendekatan ini umum dijumpai pada aplikasi PHP Native tradisional berskala kecil karena kemudahannya untuk dibuat secara cepat.

1.2 Konsep 3-Layer Web Application Architecture

3-Layer Web Application Architecture membagi sistem menjadi tiga lapisan (*layer*) logis yang memiliki tugas dan tanggung jawab spesifik serta saling berkomunikasi melalui antarmuka yang terdefinisi dengan jelas:

- **Presentation Layer (Lapisan Antarmuka / Frontend):** Bertanggung jawab menampilkan informasi kepada pengguna dan menerima masukan pengguna menggunakan HTML, CSS, dan JavaScript. Layer ini tidak boleh mengakses database secara langsung.
- **Application / Business Logic Layer (Lapisan Logika Bisnis / Backend):** Menangani aturan bisnis utama, validasi kelengkapan dan keunikan data, pengambilan keputusan, serta menyediakan endpoint **REST API (JSON)** bagi Presentation Layer.
- **Data Layer (Lapisan Pengelolaan Data / Database Access):** Bertanggung jawab menyimpan, mengambil, memperbarui, dan menghapus data di database MySQL melalui mekanisme akses data yang terisolasi (*Repository Pattern* & PDO).

BAB 2. PERANCANGAN DATABASE DAN DIAGRAM ARSITEKTUR

2.1 Spesifikasi Fitur Studi Kasus: Customer Management System

Sesuai spesifikasi studi kasus, aplikasi Customer Management System dirancang untuk mengelola data pelanggan perusahaan dengan enam fitur wajib: (1) **Tambah pelanggan**, (2) **Tampil pelanggan**, (3) **Edit pelanggan**, (4) **Hapus pelanggan**, (5) **Pencarian** berdasarkan nama atau email, dan (6) **Validasi** kelengkapan input serta keunikan alamat email.

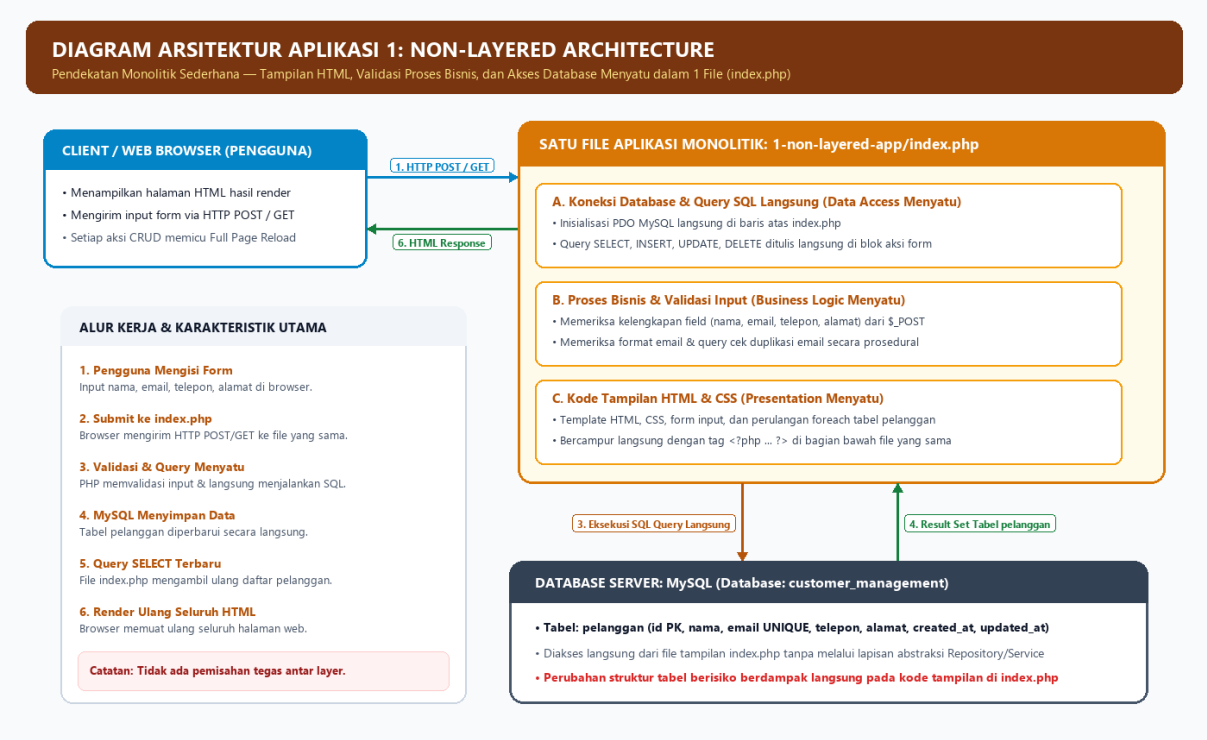
2.2 Skema Database SQL (database/customer_management.sql)

```
CREATE DATABASE IF NOT EXISTS `customer_management` DEFAULT CHARACTER SET utf8mb4;
USE `customer_management`;

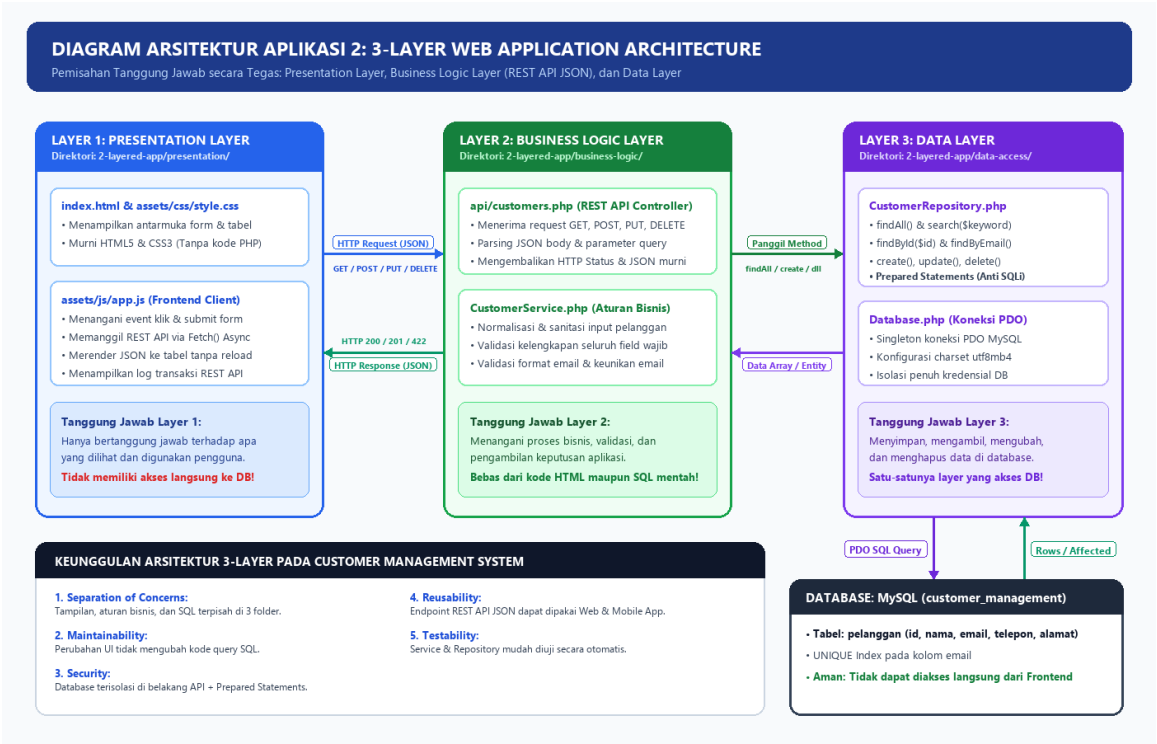
CREATE TABLE `pelanggan` (
  `id` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `nama` VARCHAR(100) NOT NULL,
  `email` VARCHAR(120) NOT NULL,
  `telepon` VARCHAR(30) NOT NULL,
  `alamat` TEXT NOT NULL,
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_pelanggan_email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

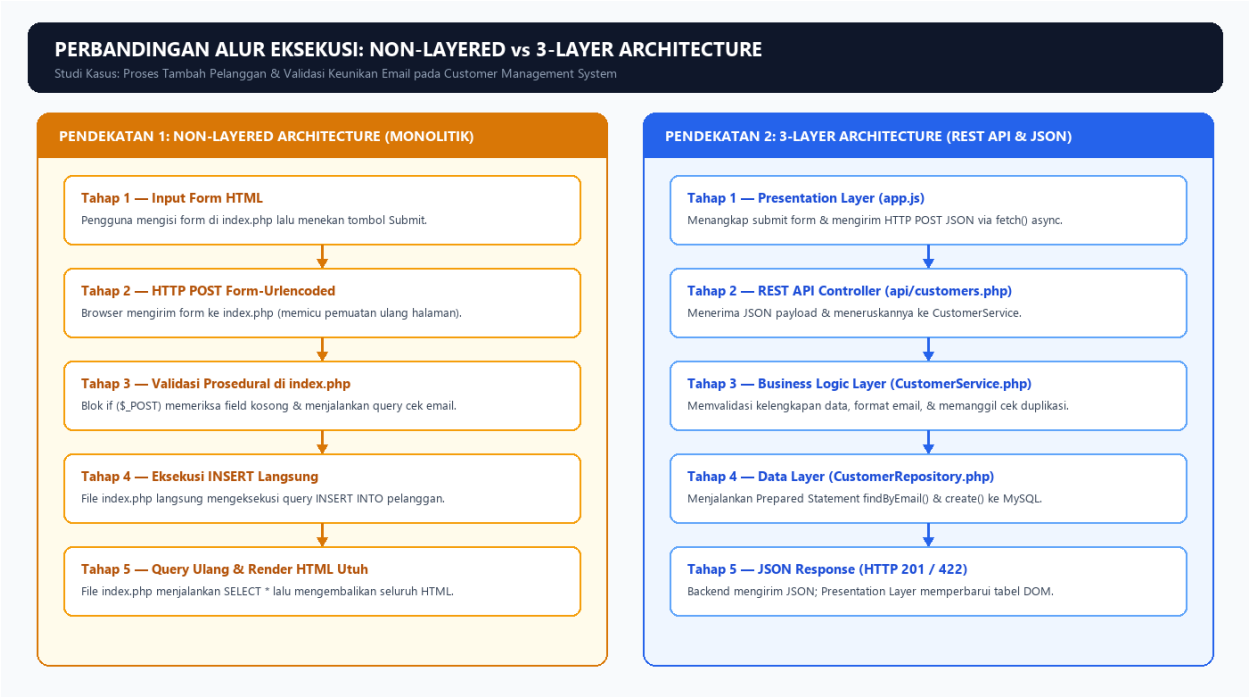
2.3 Diagram Arsitektur Kedua Aplikasi



Gambar 2.1 Diagram Arsitektur Aplikasi 1: Non-Layered Architecture (Monolitik 1 File index.php)



Gambar 2.2 Diagram Arsitektur Aplikasi 2: 3-Layer Web Application Architecture (REST API & JSON)



Gambar 2.3 Perbandingan Alur Eksekusi Tambah Data & Validasi pada Non-Layered vs 3-Layer Architecture

BAB 3. IMPLEMENTASI KEDUA APLIKASI (KODE SEDERHANA & MUDAH DIPAHAMI)

3.1 Implementasi Aplikasi Pertama: Non-Layered Architecture (1-non-layered-app/index.php)

Sesuai ketentuan Halaman 14, 15, dan 20 pada materi *3-Layer Web Application Architecture.pdf*, Aplikasi Pertama dibangun menggunakan pendekatan **Non-Layered (Monolitik 1 File)**. Seluruh kode dibagi menjadi 3 bagian di dalam satu file 1-non-layered-app/index.php:

- **Bagian 1 (Koneksi Database):** Membuka koneksi PDO ke MySQL customer_management langsung di baris atas file.
- **Bagian 2 (Proses Bisnis & Query CRUD):** Menangani hapus data ($_GET['hapus']), ambil data edit ($_GET['edit']), validasi kelengkapan & keunikan email saat form dikirim ($_POST), serta query pencarian nama/email ($_GET['cari']).
- **Bagian 3 (Tampilan HTML):** Menampilkan form input pelanggan dan tabel data pelanggan menggunakan perulangan foreach langsung di bawah kode PHP.

```
// Cuplikan 1-non-layered-app/index.php (Validasi & Query Menyatu dalam 1 File)
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if ($nama === '' || $email === '' || $telepon === '' || $alamat === '') {
        $pesan_error[] = 'Semua kolom wajib diisi.';
    }
    $stmtCek = $koneksi->prepare('SELECT id FROM pelanggan WHERE LOWER(email) = ?');
    $stmtCek->execute([$email]);
    if ($stmtCek->fetch()) {
        $pesan_error[] = 'Email sudah digunakan oleh pelanggan lain!';
    }
    if (empty($pesan_error)) {
        $stmt = $koneksi->prepare('INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?, ?, ?, ?)');
        $stmt->execute([$nama, $email, $telepon, $alamat]);
    }
}
```

3.2 Implementasi Aplikasi Kedua: 3-Layer Architecture (2-layered-app/)

Sesuai ketentuan Halaman 21 pada materi perkuliahan, Aplikasi Kedua memisahkan kode secara tegas ke dalam **tiga lapisan (layer)** dan menggunakan **REST API serta JSON** untuk komunikasi antara Frontend dan Backend:

- **Layer 1 — Presentation Layer (2-layered-app/presentation/):** Berisi index.html, assets/css/style.css, dan assets/js/app.js. Hanya bertugas menampilkan form/tabel kepada pengguna dan memanggil REST API menggunakan fetch() dengan format data **JSON**.
- **Layer 2 — Business Logic Layer (2-layered-app/business-logic/):** Berisi CustomerService.php (tempat seluruh aturan bisnis, validasi kelengkapan, dan validasi keunikan email) serta api/customers.php (endpoint REST API yang menerima request GET, POST, PUT, DELETE dan mengirim respons JSON).
- **Layer 3 — Data Layer (2-layered-app/data-access/):** Berisi Database.php dan CustomerRepository.php. Hanya bertugas menjalankan query SQL (findAll, search, findById, findByEmail, create, update, delete) ke database MySQL.

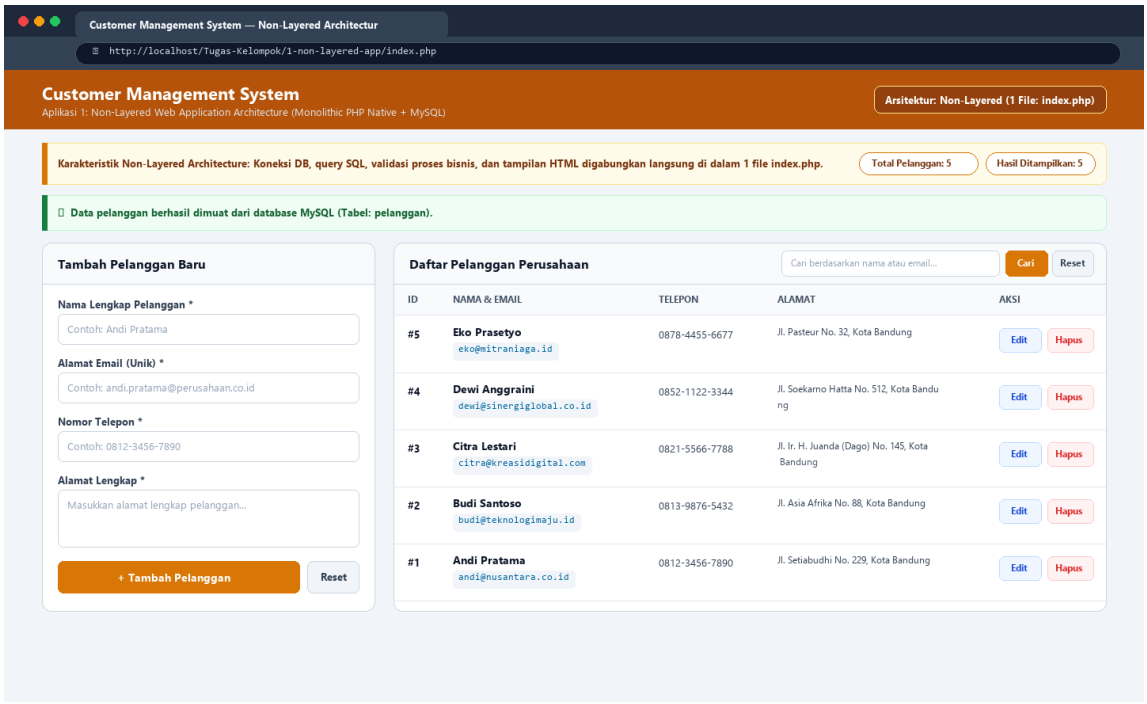
```
// 1. Presentation Layer (app.js) -> Kirim JSON ke REST API
const response = await fetch('../business-logic/api/customers.php', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ nama, email, telepon, alamat })
});
```

```
// 2. Business Logic Layer (CustomerService.php) -> Validasi Aturan Bisnis
public function createCustomer($input) {
    $errors = $this->validasiData($input); // Cek kelengkapan & keunikan email
    if (!empty($errors)) return ['status_code' => 422, 'success' => false, 'errors' => $errors];
    return ['status_code' => 201, 'success' => true, 'data' => $this->repo->create($input)];
}

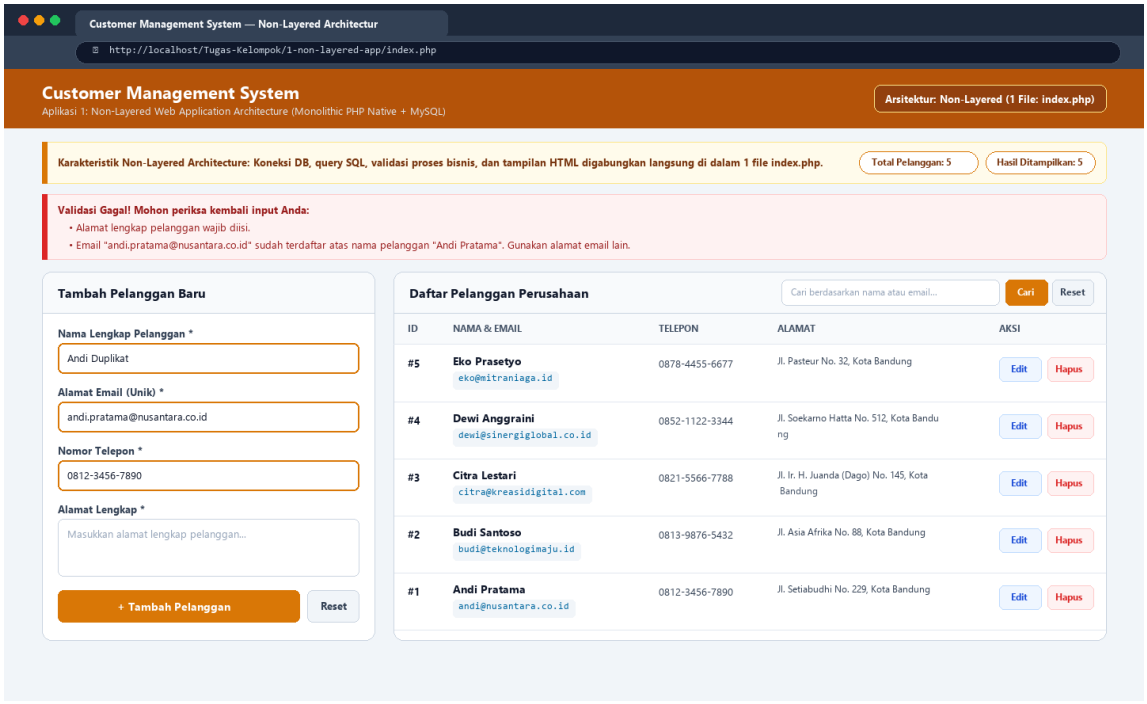
// 3. Data Layer (CustomerRepository.php) -> Eksekusi Query SQL ke MySQL
public function create($data) {
    $stmt = $this->db->prepare('INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?, ?, ?, ?)');
    $stmt->execute([$data['nama'], $data['email'], $data['telepon'], $data['alamat']]);
}
```

BAB 4. DOKUMENTASI TAMPILAN (SCREENSHOT KEDUA APLIKASI)

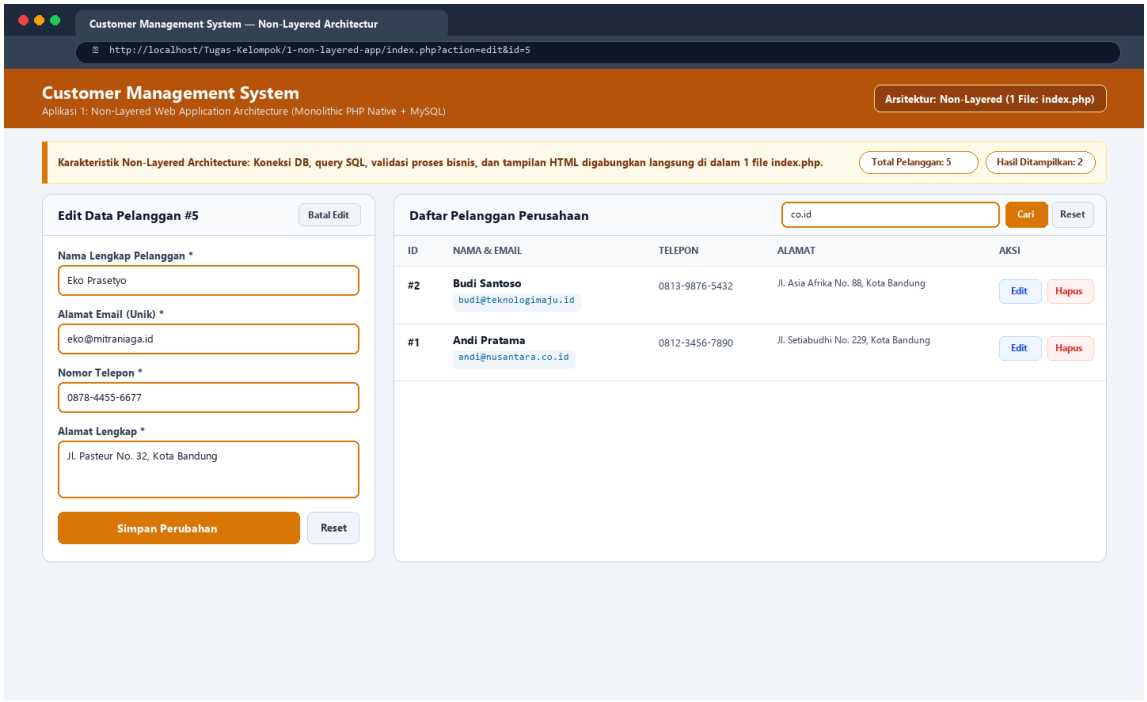
4.1 Screenshot Tampilan Aplikasi 1 (Non-Layered Architecture)



Gambar 4.1 Tampilan Utama & Daftar Pelanggan pada Aplikasi 1 (Non-Layered Architecture)

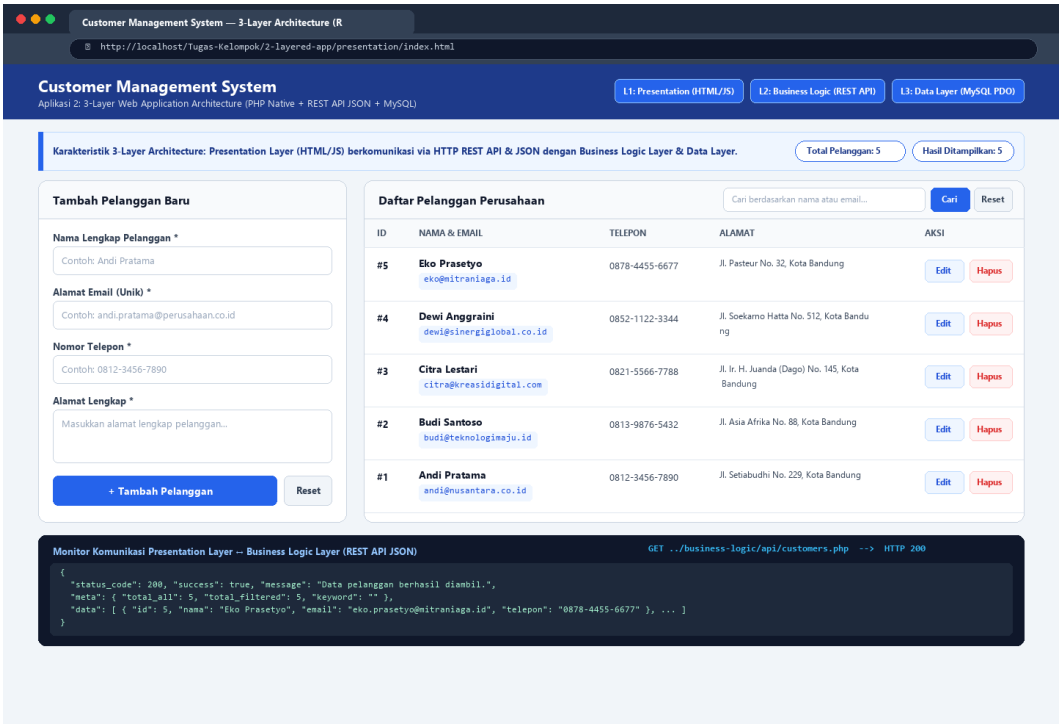


Gambar 4.2 Tampilan Validasi Kelengkapan & Keunikan Email pada Aplikasi 1 (Non-Layered Architecture)

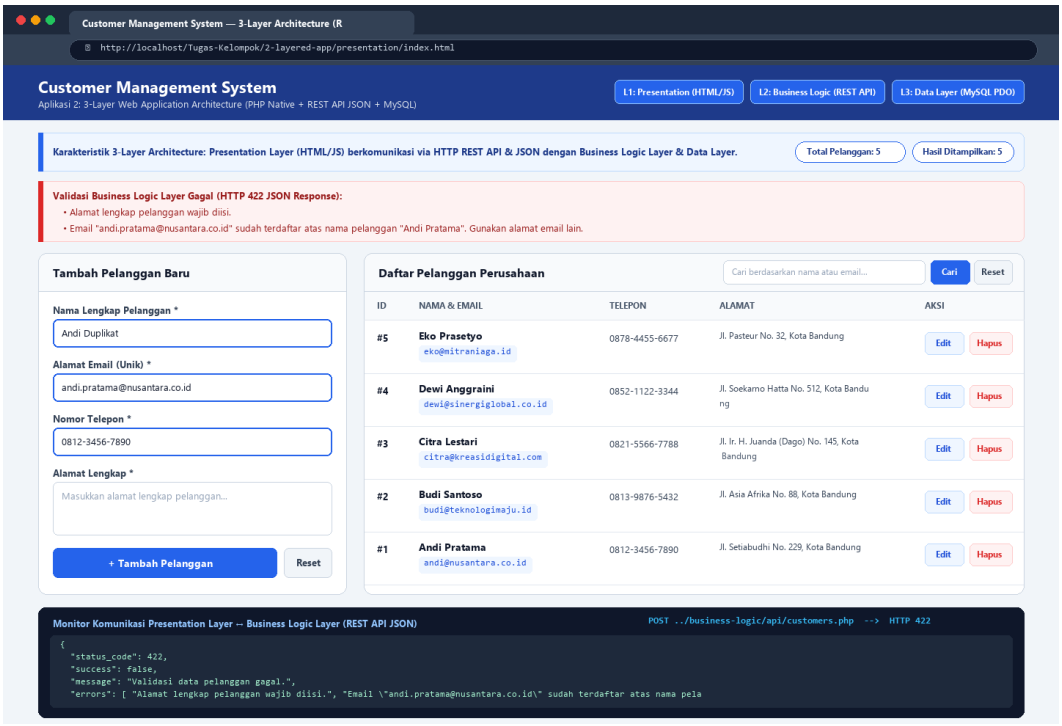


Gambar 4.3 Tampilan Fitur Edit & Pencarian Pelanggan pada Aplikasi 1 (Non-Layered Architecture)

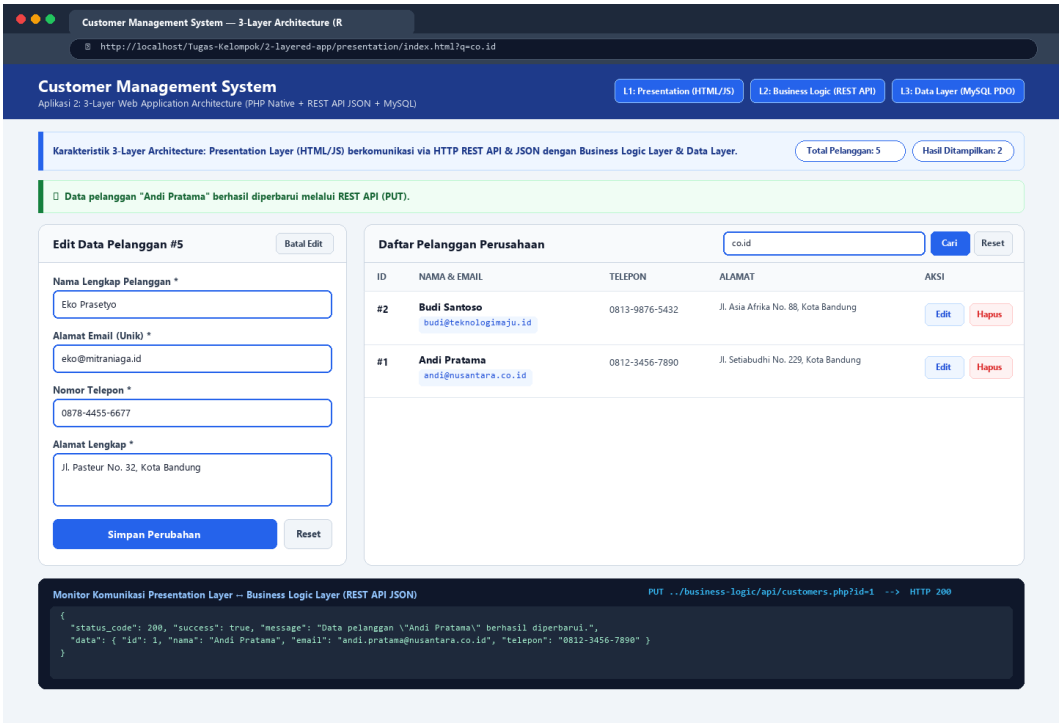
4.2 Screenshot Tampilan Aplikasi 2 (3-Layer Architecture dengan REST API & JSON)



Gambar 4.4 Tampilan Utama Aplikasi 2 (3-Layer Architecture) beserta Panel Monitor REST API JSON (HTTP 200)



Gambar 4.5 Tampilan Validasi Business Logic Layer (HTTP 422 Unprocessable Entity JSON) pada Aplikasi 2



Gambar 4.6 Tampilan Edit (HTTP PUT) & Pencarian Pelanggan pada Aplikasi 2 (3-Layer Architecture)

BAB 5. TABEL HASIL PENGUJIAN DAN PERBANDINGAN ARSITEKTUR

5.1 Tabel Hasil Pengujian Fungsional Kedua Aplikasi

Kode	Fitur Diuji	Skenario Pengujian	Hasil Aplikasi 1 (Non-Layered)	Hasil Aplikasi 2 (3-Layer API)	Status
TC-01	**Tampil Pelanggan**	Menampilkan seluruh data pelanggan awal	Tabel HTML dirender langsung dari query SELECT di index.php	GET /api/customers.php merespons HTTP 200 JSON (5 pelanggan)	**PASS**
TC-02	**Tambah Pelanggan**	Menambahkan pelanggan baru dengan data valid & email unik	POST index.php mengeksekusi INSERT lalu redirect	POST /api/customers.php merespons HTTP 201 Created JSON	**PASS**
TC-03	**Validasi Kelengkapan**	Mengirimkan form pelanggan dengan field kosong	Menampilkan alert HTML berisi 4 pesan wajib diisi	CustomerService menolak dengan HTTP 422 JSON (4 pesan error)	**PASS**
TC-04	**Validasi Format Email**	Mengirimkan email dengan format tidak valid	Ditolak oleh filter_var() di dalam index.php	Ditolak oleh CustomerService HTTP 422 sebelum menyentuh DB	**PASS**
TC-05	**Validasi Keunikan Email**	Menggunakan email yang sudah terdaftar milik pelanggan lain	Query cek duplikat di index.php menolak penyimpanan	CustomerRepository::findByEmail mendeteksi duplikat (HTTP 422)	**PASS**
TC-06	**Pencarian**	Mencari pelanggan berdasarkan nama atau email	GET ?q=... memfilter tabel dengan reload halaman	GET /api/customers.php?q=... memfilter tabel secara asinkron	**PASS**
TC-07	**Edit Pelanggan**	Mengubah informasi nama, telepon, dan alamat pelanggan	POST form_action=update menjalankan query UPDATE	PUT /api/customers.php?id=... memperbarui data (HTTP 200)	**PASS**
TC-08	**Hapus Pelanggan**	Menghapus data pelanggan dari database	GET ?action=delete&id=... menjalankan query DELETE	DELETE /api/customers.php?id=... menghapus data (HTTP 200)	**PASS**

5.2 Tabel Perbandingan Arsitektur Non-Layered vs 3-Layer Architecture

Aspek Perbandingan	Aplikasi 1: Non-Layered Architecture	Aplikasi 2: 3-Layer Architecture
Struktur File & Direktori	Sederhana, seluruh kode berada dalam 1 file (index.php).	Terpisah tegas menjadi 3 direktori layer (presentation, business-logic, data-access).
Presentation Layer (Frontend)	Menyatu langsung dengan kode logika PHP dan query SQL.	Terpisah murni (HTML5, CSS3, JS) dan berkomunikasi via Fetch API.
Business Logic Layer	Menyatu di dalam blok pemrosesan form pada file tampilan.	Dienkapsulasi di dalam class CustomerService.php & REST API.
Data Layer (Database Access)	Koneksi PDO dan query SQL ditulis langsung di dalam index.php.	Diisolasi di dalam Database.php dan CustomerRepository.php.
Komunikasi Antar Komponen	Submit form HTML standar dengan *Full Page Reload*.	Komunikasi asinkron menggunakan **REST API dan JSON**.
Pemeliharaan (Maintainability)	Lebih sulit dipelihara jika ukuran kode dan kompleksitas bertambah.	Sangat terstruktur; perubahan tampilan tidak mengganggu query database.

Aspek Perbandingan	Aplikasi 1: Non-Layered Architecture	Aplikasi 2: 3-Layer Architecture
Pengembangan Fitur (Scalability)	Cepat untuk aplikasi sangat kecil, namun rentan menjadi *spaghetti code*.	Sangat mudah dikembangkan secara modular dan dikerjakan paralel oleh tim.
Reusabilitas (Reusability)	Rendah; tidak dapat langsung dikonsumsi oleh aplikasi Mobile.	Tinggi; REST API JSON dapat digunakan oleh Web, Android/iOS, maupun sistem lain.
Pengujian (Testability)	Cenderung sulit diuji secara terpisah (terikat dengan output HTML).	Sangat mudah diuji secara otomatis per layer (seperti pada run_tests.php).

BAB 6. ANALISIS PERTANYAAN DISKUSI DAN KESIMPULAN

6.1 Jawaban Pertanyaan Analisis (Berdasarkan Materi Perkuliahan)

1. Mana yang lebih mudah dibuat?

Untuk aplikasi berskala sangat kecil atau prototipe awal, **Aplikasi Non-Layered Architecture lebih mudah dan lebih cepat dibuat** karena pengembang cukup menulis koneksi database, validasi input, query SQL, dan tampilan HTML di dalam satu file (index.php) tanpa perlu mendesain kontrak REST API JSON maupun membagi class ke dalam beberapa layer.

2. Mana yang lebih mudah diperbaiki ketika terjadi kesalahan (bug)?

Aplikasi dengan 3-Layer Architecture jauh lebih mudah diperbaiki ketika terjadi kesalahan. Karena adanya pemisahan tanggung jawab (*Separation of Concerns*), lokasi kesalahan dapat langsung diisolasi: kesalahan tampilan dicari di presentation/, kesalahan aturan validasi bisnis dicari di CustomerService.php, dan kesalahan query SQL dicari di CustomerRepository.php.

3. Mana yang lebih mudah dikembangkan ketika fitur bertambah?

Aplikasi 3-Layer Architecture jauh lebih mudah dikembangkan ketika fitur bertambah. Penambahan fitur baru (misalnya ekspor laporan, kategori pelanggan, atau integrasi aplikasi mobile) dapat dilakukan secara modular pada masing-masing layer tanpa merusak kode yang sudah berjalan.

4. Apa risiko jika seluruh kode ditempatkan dalam satu file?

Risiko utama menempatkan seluruh kode dalam satu file meliputi: (a) **Spaghetti Code** yang sulit dibaca dan dipahami, (b) **Risiko Efek Samping Tinggi** di mana perubahan kecil pada desain HTML dapat tidak sengaja merusak logika validasi atau query SQL, (c) **Konflik Kolaborasi Tim (Merge Conflict)** ketika beberapa programmer mengedit satu file yang sama, (d) **Tidak Reusable** karena logika bisnis menghasilkan HTML sehingga tidak bisa dipakai oleh aplikasi Mobile, dan (e) **Sulit Diuji (Low Testability)** secara otomatis.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Aplikasi tanpa 3-layer bukanlah berarti buruk pada setiap situasi; untuk skrip utilitas internal yang sangat sederhana, *Minimum Viable Product* (MVP) kilat, atau aplikasi dengan masa pakai singkat dan fitur minimal, pendekatan Non-Layered justru efisien karena menghindari kompleksitas berlebih (*over-engineering*). Namun, untuk sistem informasi perusahaan yang terus berkembang seperti *Customer Management System*, **3-Layer Architecture** adalah pilihan yang jauh lebih unggul dan berkelanjutan.

6.2 Kesimpulan

Berdasarkan hasil implementasi dan pengujian pada studi kasus **Customer Management System**, baik pendekatan **Non-Layered Architecture** maupun **3-Layer Architecture** berhasil menjalankan seluruh enam fitur wajib (Tambah, Tampil, Edit, Hapus, Pencarian, dan Validasi Kelengkapan/Keunikan Email) dengan tingkat keberhasilan pengujian 100% (8 dari 8 skenario uji lulus). Namun demikian, **3-Layer Web Application Architecture** terbukti memberikan struktur kode yang jauh lebih rapi, aman, mudah dipelihara, mudah diuji secara otomatis, serta memiliki *reusability* tinggi berkat komunikasi **REST API dan JSON** antara Presentation Layer dan Business Logic Layer.